

High Performance FPGA-based Scatter/Gather DMA Interface for PCIe

Hossein Kaviani-pour, *Member, IEEE* and Christian Bohm, *Senior Member, IEEE*

Abstract—Today, Data Acquisition Systems often use PCIe to communicate between FPGA based processing boards and the host computer and requires transfer of large amounts of data through this channel. But the limited services offered by the FPGA in a standalone slave mode is often a bottleneck, forcing many to use embedded systems on the FPGA, with negative impacts on the performance and available area on the FPGA as consequences. A standalone design however could manage the transfer with higher throughput and smaller footprint on the FPGA if it uses DMA and especially bus master DMA. In this paper, we present a solution to facilitate this task with high efficiency, speed, flexibility and reliability. On the hardware side, our solution uses a bus master DMA design with a 4-lane generation 2 PCIe link and on the software side, using modern Linux kernels, it supports scatter/gather vectored synchronous/asynchronous write, streaming mapping, Message Signaled Interrupt and race condition avoidance mechanism. The software has been verified on several kernels and the hardware has been tested on different FPGAs. The suite has been developed with flexibility and modularity in mind so that it can easily be adapted to different experiment. Currently it is used in the Track Engine Trigger for the IceCube Neutrino Observatory (South Pole) and is also used for command and data transfer in the development of a new high energy physics readout system.

I. INTRODUCTION

THE issue we are addressing is how to send data between a PC and a FPGA with sufficient performance measured in both the data throughput and the impact on the physical setup. This situation arises for instance whenever you want to move CPU intensive calculations to a FPGA and perform hardware acceleration or when you need to use the native parallelism nature of the FPGA for real time calculations. Regardless of the application, the setup is always the same. The FPGA, mounted on an external board, can either be connected directly to the motherboard of the PC (see fig.1) or be connected by a cable to one of the available interfaces of the motherboard. These boards are available from multiple vendors.

When connected inside the PC box, the most convenient way of connecting the FPGA board is through the PCIe. Other connections types such as Ethernet or MultiGigabit Ethernet are only more relevant when the PCB is mounted outside the PC box.

This setup can be seen as consisting of a driver domain where a driver is in charge of the communication, a bus domain



Fig. 1. FPGA PCB, mounted directly into a host PC.

and a FPGA domain(see fig.2).

Driver	Bus	FPGA
Minimum load on Host	Max throughput	Minimum resource utilization
Read,Write, Writev,iocli, Aio_write, tests	High throughput	Small DMA, interface logic, testbench

Fig. 2. The three domains in the setup with the aim and achievement in each domain.

In many applications, such as our main application, the PC is already running CPU intensive work and little resources are available to the driver. Therefore when designing the driver it is important to strive for stability, portability and low CPU load. In the bus domain, the main aim was to get maximum throughput but this requirement put pressure on the both the driver domain and FPGA domain and therefore a balance had to be attained. In the FPGA domain minimum amount of resources had to be used for the transfer mechanism since the rest was needed for the user logic. In most FPGA designs, embedded systems are used were a CPU, running an operating

Manuscript received Nov 16, 2012.

Hossein Kaviani-pour is with the Department of Physics, Stockholm University, SE 106 91 Stockholm, Sweden, (phone: +46855378699, e-mail: kaviani@fysik.su.se).

Christian Bohm is with the Department of Physics, Stockholm University, SE 106 91 Stockholm, Sweden, (phone: +46855378696, e-mail: bohms@fysik.su.se).

system, and connected to the bus interface and user logic through an internal bus structure, supervises the data transfer. In such systems, memory units and DMAs are also used. The drawback of such systems however is the large amount of on-chip resources needed for this infrastructure. Another approach would be to use a so called standalone system where a controller, in our case a DMA controller, supervises the transfer between the external bus interface and user logic (see fig.3).

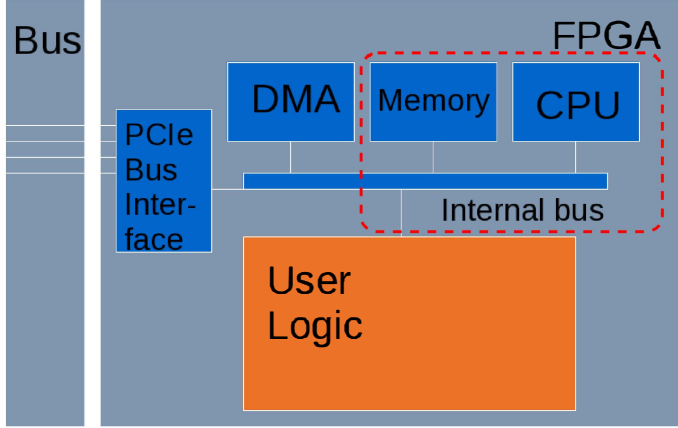


Fig. 3. The marked area shows the extra infrastructure required in an embedded system compared to a standalone system.

The PCI Express bus has a hierarchy where a Root Complex connects to one or several PCIe Endpoints, either directly or through a switch, all over the PCIe point-to-point link [1]. The Root Complex is also responsible for connecting the bus hierarchy to the CPU and main memory(see fig.4).

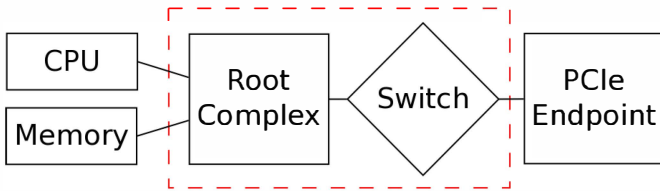


Fig. 4. PCIe bus hierarchy. The marked area are built in the chip of the motherboard of the PC.

In the FPGA the PCIe bus interface is the PCIe Endpoint whereas the Root Complex and the Switch are built into the chip of the motherboard of the PC. Through a Virtual Channel mechanism, up to 8 devices can be registered on each endpoint and operate in parallel. By using Traffic Class tags on data packages, the switch can map devices on different channels (see fig.5).

This mechanism is used when several DMA controllers are registered on the bus and are operated simultaneously. Since setting up each DMA operation takes a certain amount of time, using this mechanism reduces the idle time of the system and enables an asynchronous transfer. However, each channel requires a unique set of flow control counters and its own buffers on the Switch and Root Complex fabric. More over the extra logic needed for each DMA controller on the FPGA, makes this mechanism not suitable for low resource designs.

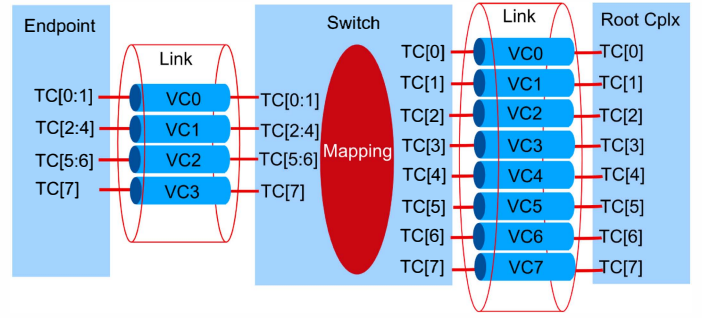


Fig. 5. An example setup of Virtual Channels. Four channels are setup between the endpoint and chipset but internally in the chipset, 8 channels are used.

In the driver domain, drivers often are required to handle multiple buffers on multiple locations at the same time or handle single buffer of non-contiguous memory. Most applications buffer the data on contiguous memory before performing the DMA transfer but some applications use a so called Scatter/Gather mechanism in combination with direct input/output(directIO) to deal with such situations. The basic idea here is that data is transferred between non-contiguous memories by multiple read or write operations, performed on smaller sections of contiguous memory instead. When designing a system that supports directIO Scatter/Gather functionality, one has to consider a few aspects. The first deals with the size of the buffers(in the case of multiple buffers) or memory segments(in the case of non-contiguous buffers). If the size of the buffers is small, then the highest throughput is achieved by merging the buffers into a contiguous buffer before using the DMA. It is only when buffer sizes are large that the cost of mapping overhead required is worthwhile. The second aspect is that the buffers are required to be page-aligned. This method is most efficient when used in an asynchronous manner but then also requires data reordering in the FPGA.

In Linux, there are two approaches to asynchronous I/O (AIO): the Linux native libaio and the POSIX AIO. In POSIX AIO, the traffic management is handled in a thread-based setup, entirely in user space. Therefore it is only virtually asynchronous and the load on the CPU increases with the number of the buffers. The libaio functions are performed in kernel space but lack the maturity POSIX AIO has. There are no gcc wrappers for the functions and the vectored version is not fully implemented yet.

II. DESIGN

The requirements for our main application was on the driver domain to have a single thread PCIe device driver with vectored write support, on the bus domain to have a minimum transfer rate of 60MB/s and on the FPGA domain have a minimal DMA controller that could operate in 250MHz and above 50% resource utilization level. Thus we decided on a standard PCI driver with single Word read/write (IOCTL), single DMA-buffer read/write(Read/Write), multiple DMA-buffer write(Writev) and asynchronous Scatter/Gather write

functionality(Aio_write), a 4-lane, generation 2, 64b PCIe link and a single engine DMA with FIFO interface to user logic.

The single word read/write is performed as a standard 32b memory read or write according to the PCIe standard[1]. The single DMA-buffer reads/writes are performed as blocking transfers. For instance during a write operation the user application puts data in a buffer and sends a standard write request to the driver which in turn buffers the data in contiguous memory. Thereafter, the driver initiates the DMA and starts it and then goes to sleep. When the DMA is done, an interrupt is sent to the driver where the interrupt subroutine will wake up the sleeping process. The result of the transfer is then checked and sent back to the user application.

The vectored write, which is put in a function library and operates in user space, merges the buffers into a single buffer and uses the ordinary write to transfer the data. This is since in our main application each buffer is only 12B large. Due to the same reason we do not use directIO. However by using the asynchronous interface of the libaio library, we can initiate multiple write operations at once and avoid having a context switch for every operation. Thus again by using functions put in our function library, io_submit is used to send multiple (asynchronous) write operations. The driver handles these requests with its Aio_write interface and performs the transfers in a synchronous fashion.

III. FIRMWARE

The firmware is based on the Xilinx Bus Master DMA and is optimized to have minimal resource utilization. The interface to the user logic is through FIFOs and the interface to the PCIe bus is through the Xilinx Endpoint[2].

For the 7-series of Xilinx FPGAs, the Endpoint had to be regenerated and modified. The new endpoints are twice as large as the previous ones (measured in flip-flops and Block RAMs)[2], [3] and support the AMBA AXI4 interface[4] exclusively.

IV. SOFTWARE

The driver is designed for common Linux versions (verified for 2.6.35) according to the standards [5]. It uses the Message Signaled Interrupt mechanism to receive interrupts since it is the native PCIe capability and is to be preferred. It uses streaming mapping in order to increase performance.

V. TESTS

The setup has been tested on a Gigabyte GA-H55M-UD2H motherboard with an Intel i3-530 processor and 4GB DDR2 memory using the Linux kernel 2.6.34.9-69 (Fedora13). The main FPGA board used in test and development was the HTGlobal Virtex-6(-2) evaluation board but the setup has been verified on a Xilinx VC707 evaluation board (Virtex-7), a Xilinx KC705 evaluation board (Kintex-7) and a Xilinx ML605 evaluation board (Virtex-6). The result was compared to the MPRACE framework running on a Virtex-5 [7]. The DMA controller functionality has been tested using Chipscope Pro [6], an on-chip logic analyzer firmware, organized in a star-formation, with small data modules and central trigger module.

VI. RESULTS

Fig.6 shows the result for the tested FPGA boards. The highest achieved transfer rate for single buffer transfer was 784MBytes/s on read and 748MBytes/s on write transactions for the Virtex-7 evaluation board. The theoretical limit of a 4-lane generation 2 PCIe bus is according to our calculations roughly 1500MB/s. This is because of the encoding and signaling overhead in the Physical Link Layer and the flow control traffic of Data Link Layer. We would thus have 52% efficiency with current package sizes (8KB).

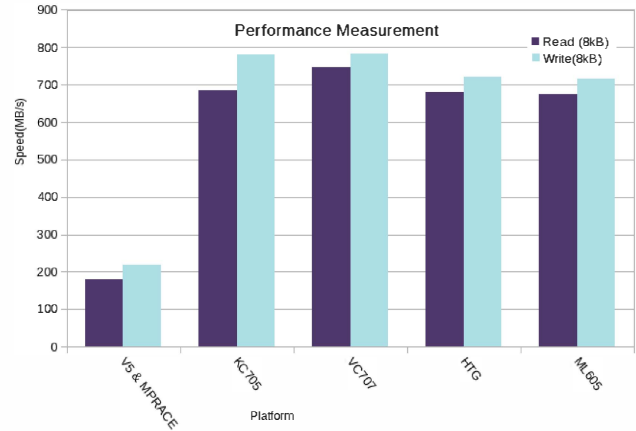


Fig. 6. Performance measurement for 8kB buffers on different FPGA boards.

Fig.7 shows the comparison between the single buffer transfer and the Aio_write transfer for 30 write operations. The achieved result was 1080MB/s with Aio_write. These tests were performed on the HTGlobal Virtex 6 FPGA card.

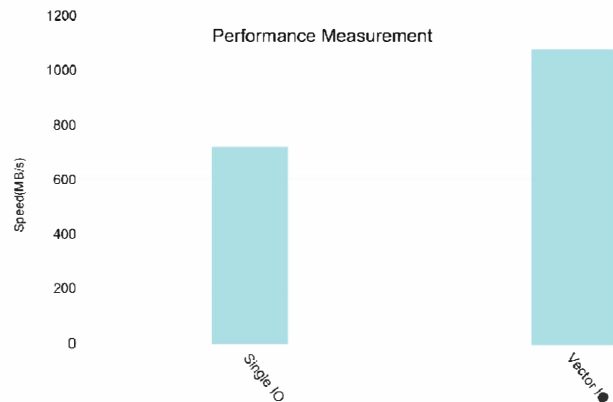


Fig. 7. Performance measurement for single transfer and Aio_write.

This setup is used as the main host-FPGA data transfer method in the Track Engine Trigger [8] for the IceCube Neutrino Observatory[9]. The driver is, however, adapted for an older version of Linux since the PC in the Track Engine is conditioned to run a 2.6.18 kernel.

The suite is also used for command transfer and the data readout during the development of the ATLAS Tile Calorimeter front-end electronics [10].

VII. OUTLOOK

A considerable performance improvement is easily achieved by increasing the buffer size in the transfer. This size is optimized for our main application. Another easy improvement is achieved by using a motherboard with a better chipset. In addition to the Switch control flow buffers, the chipset has a property called Maximum Payload Size which denotes the maximum data packet burst size allowed on the bus. On our test setup it is the lowest possible, 128B.

REFERENCES

- [1] "PCI Express® 2.0 Base Specification Revision 0.9", <http://www.pcisig.com/specifications/pciexpress/base2/>, PCI-SIG, 2006.
- [2] "LogiCORE IP Virtex-6 FPGA Integrated Block v2.5 for PCI Express", http://www.xilinx.com/support/documentation/ip_documentation/v6_pcie/v2_5/ds800_v6_pcie.pdf, Xilinx Inc., 2012.
- [3] "LogiCORE IP 7 Series FPGAs Integrated Block v1.4 for PCI Express", http://www.xilinx.com/support/documentation/ip_documentation/pcie_7x/v1_4/ds821_7series_pcie.pdf, Xilinx Inc., 2012.
- [4] "AMBA 4 AXI4-Stream Protocol", <http://www.arm.com/products/system-ip/amba/amba-open-specifications.php>, ARM Inc, 2010.
- [5] J. Corben, A. Rubini and G. Kroah-Hartman, "Linux Device Drivers", O'Reilly Media, 2005.
- [6] "Chipscope Pro Software and Cores", http://www.xilinx.com/support/documentation/sw_manuals/chipscope_pro_sw_cores_9_1i_ug029.pdf, Xilinx Inc, 2012.
- [7] G. Marcus, W. Gao, A. Kugel and R. Männer, "THE MPRACE FRAMEWORK: AN OPEN SOURCE FOR COMMUNICATION WITH CUSTOM FPGA-BASED ACCELERATORS", 2011 VII Southern Conference on Programmable Logic (SPL), 2011.
- [8] C. Bohm, H. Kavianipour, D. Nygren, C. Robson, C. Wernhoff and G. Wikström, "A Low Energy Muon Trigger for IceCube", IEEE NSS, 2008.
- [9] F. Halzen and S. R. Klein, Invited Review Article: "IceCube: An Instrument for Neutrino Astronomy", Review of Scientific Instruments, 2010.
- [10] S. Muschter, K. Anderson, C. Bohm, D. Eriksson, H. Kavianipour, M. Oreglia and F. Tang, "A Full Slice Test Version of a Tentative Upgraded Readout System for TileCal", ACES 2011 - Common ATLAS CMS Electronics Workshop for SLHC, 2011.
- [11] "Intel® 5 Series Chipset and Intel® 3400 Series Chipset Datasheet", <http://www.intel.se/content/www/se/sv/chipsets/5-chipset-3400-chipset-datasheet.html>, Doc.No.: 322169-004, Intel Corp., 2012.